University of South Florida

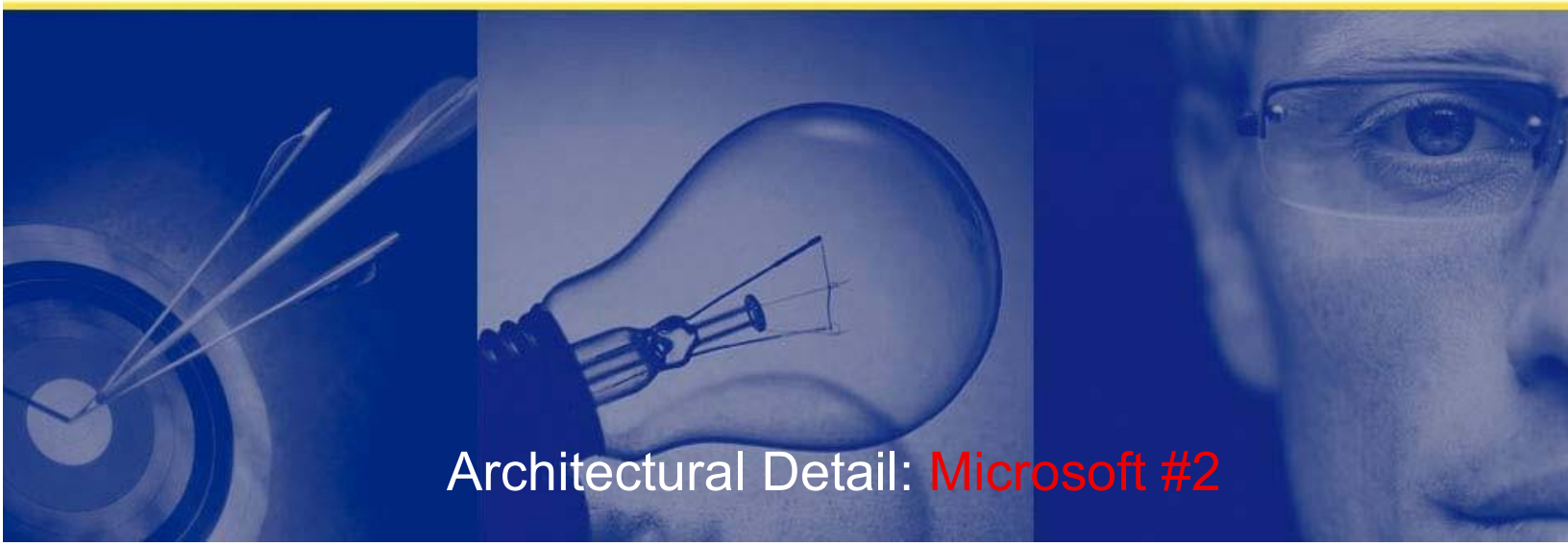
Bellini College of AI, Cybersecurity and Computing

Detailed Design Document

CIS-4910 Senior Project Fall 2026

Microsoft 2, Team 3

Bruno Rocha, Kapil Srikanth, Ross Volenec, Joseph Bressani



Document Control

Document Version History			
Date	Version #	Author	Change reference
15-October 26	1.4	Team 3	Added Public Health, Safety, and Welfare section.
15-October 26	1.3	Team 3	Rubric Review
15- October 26	1.2	Team 3	Added behavioral view; expanded logical, physical, and data view
13-October 26	1.1	Team 3	Updates from Oct. 9 Sponsor Meeting
7-October 26	1.0	Joseph Bressani	Initial DDD draft
7-October 26	1.0	Kapil Srikanth	Initial DDD draft
7-October 26	1.0	Bruno Rocha	Initial DDD draft
7-October 26	1.0	Ross Volenec	Initial DDD draft

Document Reviewers / Team Members			
Date	Version #	Name/Title	Organization
7-October 26	1.0	Joseph Bressani	University of South Florida
7-October 26	1.0	Kapil Srikanth	University of South Florida
7-October 26	1.0	Bruno Rocha	University of South Florida
7-October 26	1.0	Ross Volenec	University of South Florida

Stakeholders

Name	Title
Chad Hage	Sponsor
Brad Lawrence	Sponsor

Table of Contents

Introduction 3

Purpose 4

Sponsor and Context 4

Scope 4

Architectural Description Framework 5

Benchmark Approaches 6

Architectural Goals, Constraints and Assumptions 7

Goals 7

Constraints 7

Assumptions 8

Artificial Intelligence Impacts and Uses 9

AI Assisted Content Notation 9

Scope of Project Architecture 10

High Level Architecture (HLA) 10

Architectural Views 11

Logical 11

Physical 12

Data 12

System Configuration Details 20

System Inputs API if used 21

System Output API if Used 21

System Decomposition 22

System Coupling 22

System Cohesion 23

Application servers 23

Web servers 23

Database servers, design and Schemas 23

Storage Requirements 24
Storage layout 24
Module Specifications 25
Module Design 26
Module Processing 26
Interfaces 27
Formulas & Algorithms 27
Architecture Decision Records 32
System Size and Performance 32
Security Considerations 33
Public Health, Safety and Welfare 34
References 36

Introduction

Purpose

The Detailed Design Document (DDD) is meant to translate the System Requirement Specification (SRS) for this project into an implementable architecture. The harness provides an AI coding agent with a coding task, it will then run in an isolated sandbox and grade the results independently and publish a side-by-side comparison for different matrixes we believe most software engineers would deem valuable. These matrixes are correctness, security, code quality, token efficiency, and human help the agent's needs. This document defines the architectural style, the components and their responsibilities, the interfaces between them, the data they persist, the infrastructure they run on, and the design decisions and trade-offs behind each choice.

Sponsor and Context

The project is sponsored by Microsoft (Chad Hage, Brad Lawrence). The group must decide which AI coding agent to adopt and has no objective, repeatable way to compare the candidates. The initial agents are OpenAI Codex, Anthropic Claude Code, GitHub Copilot, and Google Antigravity. Every agent runs against the shipped default settings, so differences in the results come from the agent's tooling, not the model.

Scope

In scope:

- A standard reference application with automated tests (F1).
- Isolated, credential-free execution of agent-generated code (F2).
- Orchestration of agent runs under turn, time, token, and cost limits (F3).
- Independent grading, including secret and license scans (F4).
- Per-run telemetry and public results microsite (F5).

Out of scope:

- Training or fine-tuning models.
- Comparing models against each other.
- Benchmarks other than the JavaScript reference application.
- Running the harness as a hosted service; it runs on the operator's machine.

- Deploying agent-generated code to any production system.
- Third-party extensions such as SpecKit in the baseline comparison

Architectural Description Framework

The document is an architecture description in the sense of ISO/IEC/IEEE 42010:2022 [10]. It identifies the system's stakeholders and their concerns and answers each concern with a view. The views follow Kruchten's 4+1 view model [11] (logical, physical, behavioral/process, development and scenarios) plus a data view, as required for this course.

Each stakeholder concern below is answered by at least one view or section of this document.

Stakeholder	Key concerns	Answered by
Sponsor (Chad Hage, Brad Lawrence)	Which tool pipeline is production-ready; fair, repeatable comparison; security of shipped defaults; weekly demos	Benchmark Approaches; Scenarios view; Security Considerations; C-8, C-9; ADR-01
Course instructors and peer reviewers	Rigor, SRS traceability, internal consistency, feasibility	SRS Traceability Matrix; Architecture Decision Records; all views
Team developers	Module responsibilities, interfaces, where code lives, how it is built and tested	Logical view; Interfaces; Development view; Module Specifications
Benchmark operators (team members running pilots)	Repeatable procedure, isolation of the agent, evidence that survives review	Behavioral view (Figures 5 and 8); Physical view; Scenarios view
Results readers (microsite visitors)	Honest, comparable, clearly labeled results	Data view; Figure 7; Web servers; ADR-01
USF and Microsoft compliance	No personal data or secrets; least privilege; monitored workloads	Physical view; Security Considerations; C-7

Viewpoints and notations used:

View (viewpoint)	Notation	Where
Logical	Component diagram (layered modules)	Figure 2; System Decomposition
Physical	Deployment diagram with trust zones	Figure 3; node table

Data	Entity-relationship diagram; schema tables	Figure 4; Database servers, design and Schemas
Behavioral (process)	Sequence and activity diagrams	Figures 5–8
Development	Repository and branch tables	Development view
Scenarios (+1)	Walk-throughs of Task 001 and Task 003	Scenarios view

Benchmark Approaches

The project compares tools through three approaches that share one evaluator, one evidence layout and one publication path, but are never mixed in one score (ADR-01, ADR-13).

	A. Feature matrix	B. Native-tool pilots	C. AutoGen harness
Question answered	What each tool documents it can do	How each vendor pipeline performs in its shipped defaults (sponsor's main question)	How agents perform under identical, enforced limits
Who drives the agent	No agent; reviewers read documentation	The operator, using the vendor's own CLI or IDE (Codex CLI, Claude Code, Copilot CLI, Antigravity)	The harness controller (M02) through the model adapter (M04)
Model	Not applicable	Tool's default selection, recorded per run	Pinned model profile (azure-prod)
Limits enforced by	Not applicable	The tool's own approvals and sandbox, recorded by the operator	Host-enforced turns, time, tokens and container limits (ADR-03)
Modules used	M11, M13	Procedure replaces M02–M07 (see mapping below); M08, M10 record format, M12, M13	M01–M10, M12, M13
Scored with formulas (M09)	No	No; raw suite results, time, approvals and tokens are reported	Yes
Status	Specification v0.2 [4]	Nine pilot runs complete across four tools and two tasks (Scenarios view)	Partial implementation

How Approach B maps onto the architecture. The written procedure [8], [24] performs by hand what harness modules do automatically, so the evidence it produces fits the same record and publication path:

Harness module	Approach B equivalent
M01 Contracts	Fixed starting commit, agreed evaluator commit and grader hashes recorded before the run
M03 Staging	git archive snapshot of app/ and the task file only, with no Git metadata or graders
M05 / M06 Coder loop and sandbox	The vendor tool's own agent loop, approvals and sandbox; settings recorded, not enforced
M07 Freezer	Original files copied and before/after SHA-256 manifests compared
M08 Evaluation	The same evaluator-owned suites, run outside the agent workspace
M10 Record writer	A results README plus manifests, logs and transcripts on a results branch

A native tool can move from Approach B into Approach C only through a tool adapter that reports its capabilities over interface I09 (headless mode, model pinning, sandbox routing, usage reporting). Tools that cannot meet I09 stay in Approach B.

Architectural Goals, Constraints and Assumptions

Goals

The architecture is designed to achieve the following goals:

1. **Repeatable evaluation:** Give each agent the same baseline application, task prompt, permissions, and execution limits. Reset the application before every run.
2. **Safe execution:** Isolate agent-generated code from host credentials, unrelated files, and external networks.
3. **Independent grading:** Evaluate submitted changes using protected acceptance tests and security scans that the agent cannot modify.
4. **Controlled automation:** Allow agents to generate changes, receive execution feedback, and attempt corrections within defined resource and execution budgets.
5. **Traceable results:** Record configuration, test results, security findings, token usage, execution time, and human involvement for each run. Publish comparable results through the microsite.
6. **Extensibility:** Separate agent integrations from task definitions, sandbox management, and grading so additional agents can be evaluated without redesigning the entire harness.

Constraints

ID	Constraints	Source	Architectural Impact
C-1	The harness runs on the operator's machine and not as a hosted service	SRS Scope	Local-first design: CLI Docker/embedded database; no multi-tenant web back end

C-2	Only the JavaScript reference application is benchmarked	SRS Scope	
C-3	Budget is limited to the $100 Azure credit given to us by the sponsor.	Initial Sponsor Meeting	Hard per-trial token and cost caps; small pinned model; trial count configurable. Approach B tools run on each member's own tool plan; their usage is recorded when the tool reports it
C-4	Fixed schedule based on the semester; Final Demo on Dec 4th,2026.	Course Syllabus	Limits scope of what can be done in the short amount of time provided.
C-5	Agent-generated code is never deployed to production	SRS Scope	Execution limited to disposable containers; no deployment credentials in the sandbox
C-6	Each agent must be driven non-interactively	Repeatability Goal	Only tools that meet I09 join the harness; IDE-only tools such as Antigravity stay in Approach B, where every approval is logged
C-7	USF policy, FERPA and Microsoft usage guardrails apply; sponsor workloads are monitored	Initial Sponsor Meeting	No personal data in results; least-privilege access to Azure; the public microsite contains only redacted, aggregated evidence.
C-8	Demo expected at every weekly meeting; a video linked in the GitHub issue is acceptable	Sponsor Meeting, Oct. 9th	Runs are scripted as any member can demo them.
C-9	Tools run in their shipped default configuration; extensions are excluded (e.g. SpecKit)	Sponsor Meeting, Oct. 9th	Record default model, reasoning, and approval modes.

Assumptions

ID	Assumption	If false, then…
A-1	The operator's workstation can run the Docker Engine and has a Desktop with at least 16 GB RAM and 100 GB free disk space.	Runs trials sequentially or uses an Azure VM as the operator machine
A-2	Each agent CLI (Codex CLI, Claude Code, Copilot CLI, and Antigravity CLI) supports a headless mode and either a base-URL override or reports token usage.	Adapter falls back to parsing the agent's own usage output; result would be flagged for "self-reported usage"
A-3	Published results contain no confidential information.	Publisher redaction rules block the publishing step.
A-4	Model outputs are non-deterministic; N ≥ 3 trials per agent/task give a usable estimate	Increase N for close comparisons; report confidence intervals rather than single runs

A-5	One member runs each tool end to end with identical task files and prompts	Results are not comparable; rerun from benchmark-start-v1
A-6	Each tool's models stay available through Dec. 4th	Record model version per run; a new version starts a new cohort
A-7	Each tool reports per-run tokens	Token score is dropped; totals are out of 90

Artificial Intelligence Impacts and Uses

AI Assisted Content Notation

Portions of this document were prepared with the assistance of generative AI tools. As a team, any input received from AI was reviewed, edited, and verified against the contents of the SRS, sponsor meeting notes, and the project repository. Diagrams were produced with Mermaid or Claude to ensure a more professional view compared to doing it by hand.

Section or figure	Version	AI role	Source checked against	Human judgment
Sections and figures not listed below	1.0	AI-assisted drafting and editing	SRS [2], sponsor notes [3], repository [1]	Team authored, edited and verified
Sponsor and Context; C-8, C-9; A-5–A-7; pilot paragraph; Security Considerations text and Table 21	1.1	AI-drafted	Oct. 9 meeting transcript [3]; repository	Reviewed, edited and verified by the team against the listed sources
Logical, physical and data view text; Figures 5–7	1.2	AI-drafted from existing DDD content	Sections of v1.1	Reviewed, edited and verified by the team against the listed sources
Architecture Description Framework; Benchmark Approaches; revised C-3, C-6, A-2; Figure 8; Development and Scenario views; expanded ADRs and ADR-13; Traceability Matrix; pilot evidence; Appendices A–C	1.3	AI-drafted	Results branches and procedures in [1], [8], [24]; SRS [2]	Reviewed, edited and verified by the team against the listed sources

AI coding agents are also the subject of this project: their outputs are evidence to be graded, never trusted inputs. Every agent-written file is preserved, hashed and graded by evaluator-owned suites, and the public site labels agent-generated code as AI-generated (Spec 4.4).

Scope of Project Architecture

The harness is a modular monolith organized as a layered architecture whose core workflow is a pipes-and-filters pipeline (Plan → Execute → Grade → Score → Publish). Agent integrations use a plug-in (microkernel) style: the orchestrator is the stable core, and each agent is an adapter loaded from a registry. Telemetry follows an event-based, append-only style (every stage emits events to a log that is later ingested). The results microsite follows a static-site (JAMstack) style. Each choice is tied to a requirement or constraint below; the rejected alternatives are discussed in ADR-01, ADR-05 and ADR-11.

High Level Architecture (HLA)

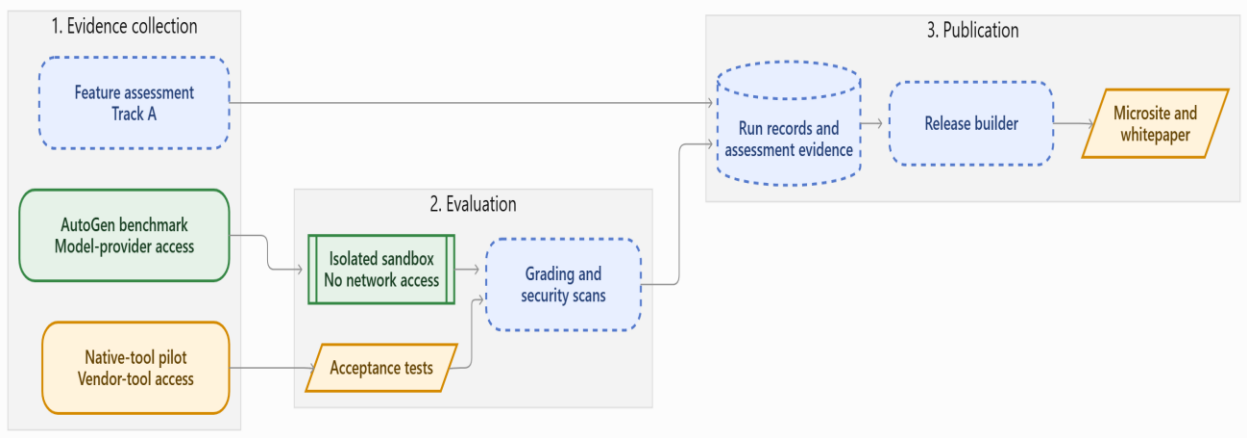


Figure 1. High-level architecture and implementation status.

Green, solid border	**Amber, solid border**	**Blue, dashed border**
Implemented	Pilot or prototype	Proposed

Figure 1. High-level architecture and implementation status. Green, solid border = implemented; amber, solid border = pilot or prototype; blue, dashed border = proposed.

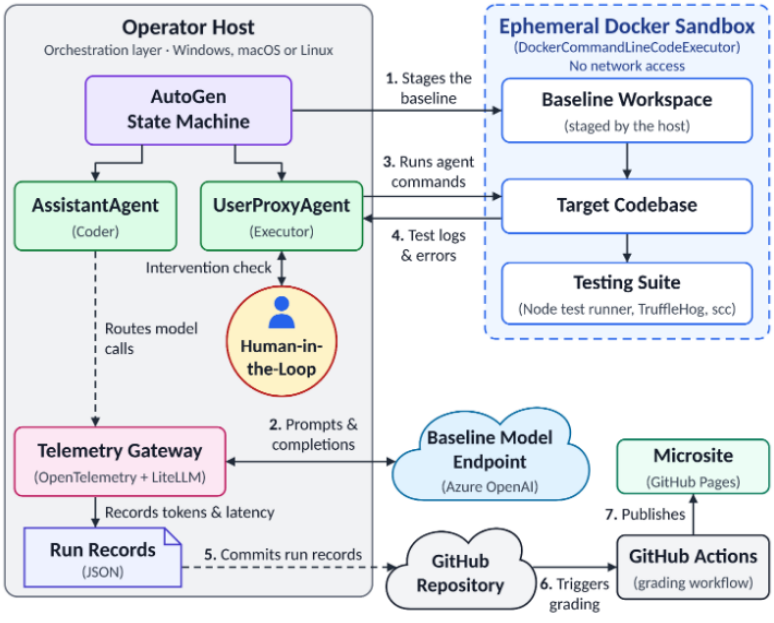
Evidence producers generate candidate code or documented findings. Candidate code is graded inside the trusted evaluation boundary. Graded results become run records that a release builder publishes. Readers cannot start running.

Architectural Views

Logical

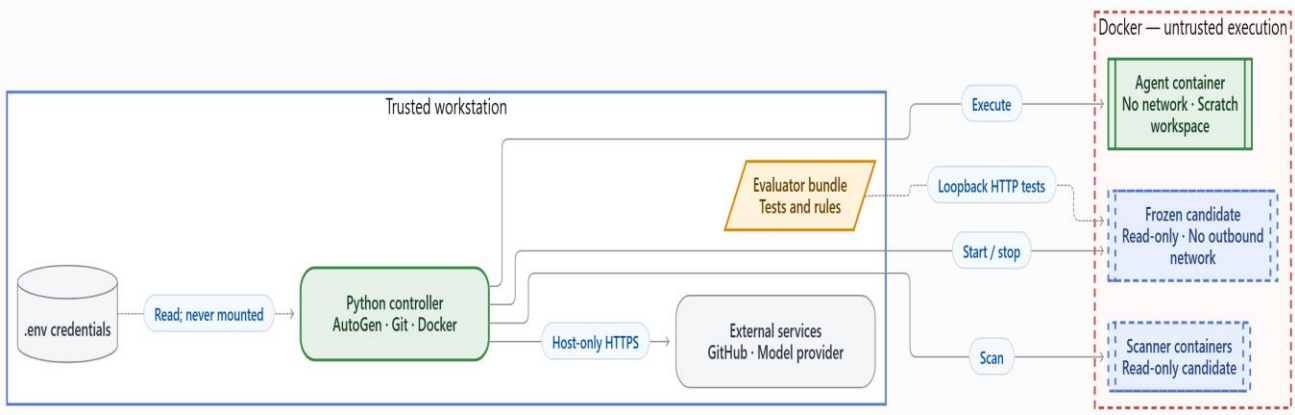
The logical view decomposes the harness into four layers, each with one responsibility; the module table under System Decomposition (M01–M13) gives the full list with code and status.

- **Contracts and control:** M01 Contracts validates the task, profile, policy and campaign; M02 Controller owns the run lifecycle, budgets and final status.
- **Execution:** M03 Staging copies the seed without credentials; M04 Model adapter records every provider call; M05 Coder loop counts turns and requests approvals; M06 Sandbox enforces container limits and teardown.
- **Evaluation:** M07 Freezer hashes the candidate and enforces the file manifest; M08 Evaluation runs evaluator-owned suites and scans; M09 Scoring applies the metric contract.
- **Records and publication:** M10 Record writer writes one validated record per run; M11 Assessment holds Track A feature-matrix cells; M12 Export and referee regrades in CI; M13 Release builder publishes reviewed records to the site.



The implementation differs from the SRS view: it uses `CodeExecutorAgent` instead of `UserProxyAgent`, wraps the model client instead of using a LiteLLM gateway, grades pull requests with code from the pull request itself, and does not publish run records.

Physical

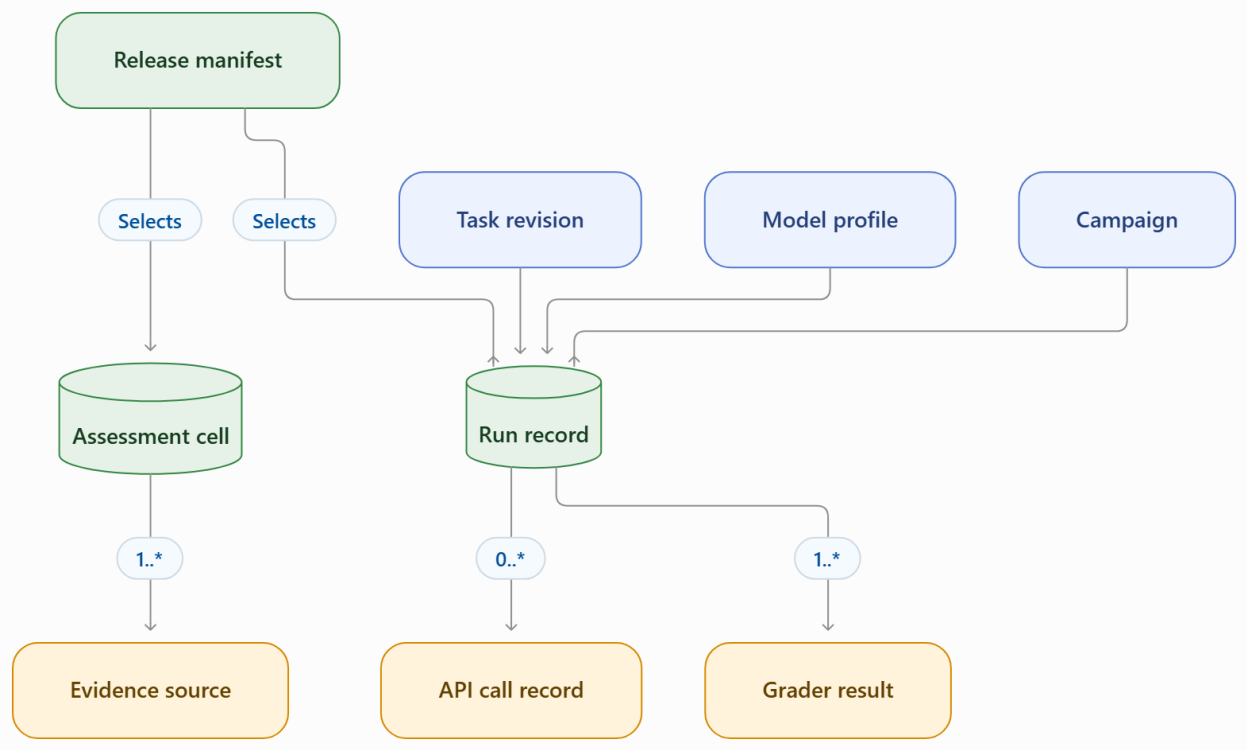


Green, solid border	**Amber, solid border**	**Blue, dashed border**
Implemented	Pilot or prototype	Proposed

The physical view places every component on one of three trust zones.

Node	Runs	Network	Holds secrets
Operator workstation (trusted)	Python controller (AutoGen, Git, Docker SDK); evaluator bundle; run records; Approach B vendor tools and their isolated snapshot folders	Host-only HTTPS to GitHub and the model provider	Yes: `.env` read by the host, never mounted into containers
Docker engine (untrusted execution)	Agent container with scratch workspace; frozen-candidate and scanner containers (proposed)	None (`--network none`); evaluator reaches the candidate over loopback only	No
GitHub (hosted)	Repository and results branches; Actions referee job; Pages static site	Public	No model credentials; the publishing job alone gets write permission (proposed, ADR-11)
Model provider (hosted)	Azure model deployment (azure-prod profile for scored runs)	Called only from the host adapter	API key stays on the host

Data



A RunRecord links one task, one profile, and one campaign to its calls and grader results. Corrections are separate records linked by `phase` and `retry_of`. Track A uses AssessmentCells. A release manifest selects reviewed records by hash.

Persistence strategy. All state is versioned JSON files; no database server runs (ADR-09). Every file is written to a temporary path, validated against JSON Schema 2020-12, then renamed, so a crash never leaves a half-written record. Raw logs and transcripts stay on the operator machine; only redacted records are committed to a results branch, and only reviewed records selected by hash in a release manifest reach the public site.

Data flow.

1. Task manifest, model profile and campaign manifest are hashed at validation (M01).
2. Each provider call and command is appended to the run's event log (M04, M05).
3. Grader results attach to the RunRecord with suite hashes (M08); scores follow (M09).
4. M10 writes one terminal RunRecord per run; corrections are new records linked by `retry_of`.
5. Reviewed records are committed to `benchmark-results/<tool>/<run>/`, then copied into `site/data/<release>/` by M13. Approach B evidence follows the same path from step 5.

Behavioral

The behavioral view shows the four workflows the system runs most: one scored harness run, the grading decision for a finished attempt, publication of results, and one native-tool pilot run.

Scored harness run (Approach C)

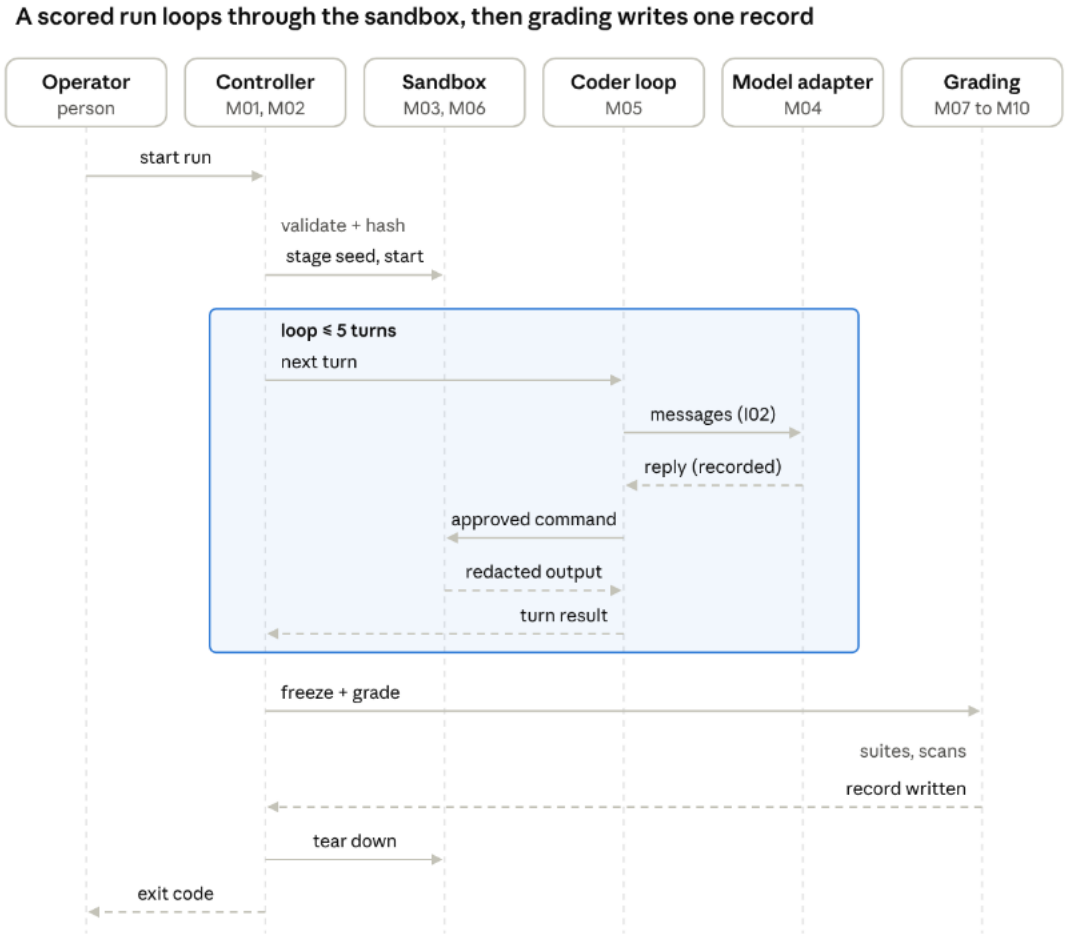


Figure 5. Sequence: one scored harness run (Approach C)

The controller validates and hashes every input before a run ID exists, then drives at most five coder turns inside a network-less container; every model call is recorded and every command result is redacted before the agent sees it. Only after the candidate is frozen do evaluator-owned suites grade it, and the run ends with exactly one record and an exit code (0 passed, 1 failed, 2 invalid input or infrastructure error, 130 cancelled).

Grading and final status

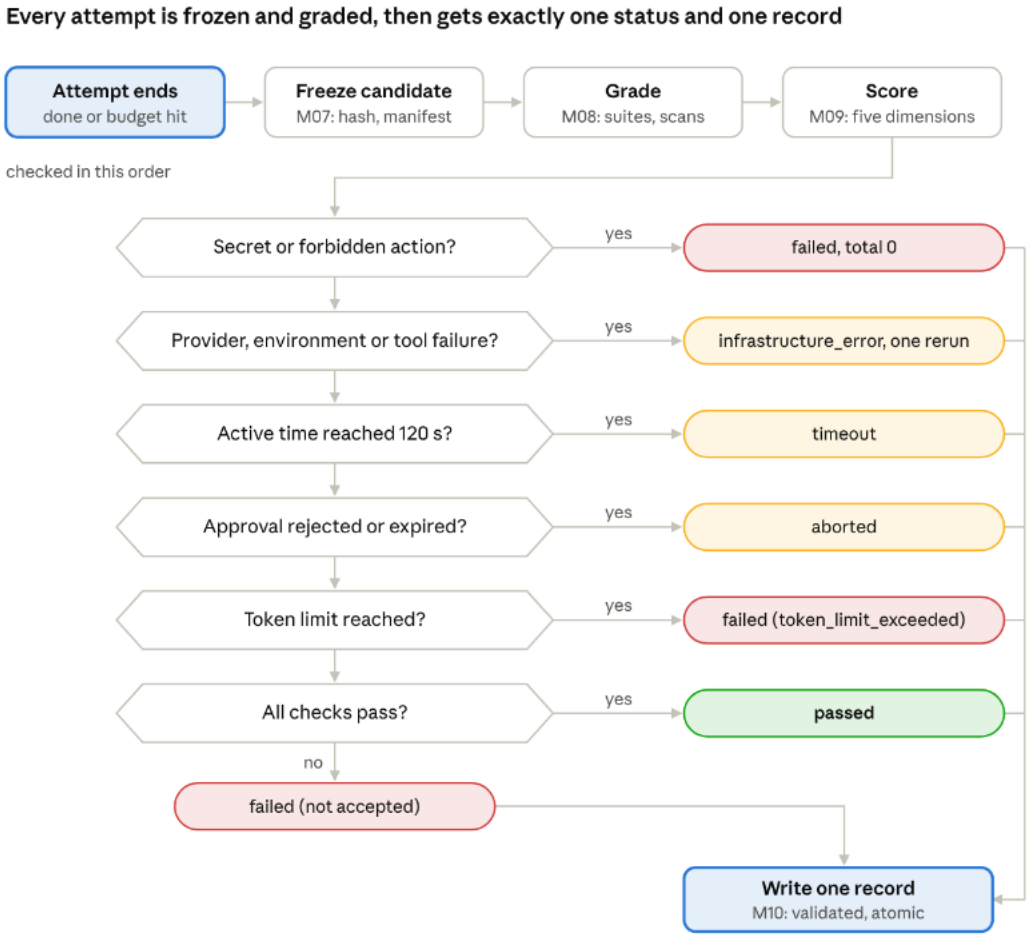


Figure 6. Activity: grading and final status of an attempt

Whether the agent finished or ran out of budget, the candidate is frozen and graded before any status is chosen. Conditions are checked in the order of the Module Processing status table, so a leaked secret or forbidden action overrides every other outcome. Every path ends in exactly one validated record. Approach B runs use the freeze and grade steps but report raw suite results instead of a status from M09.

Review, CI regrade and publication (proposed)

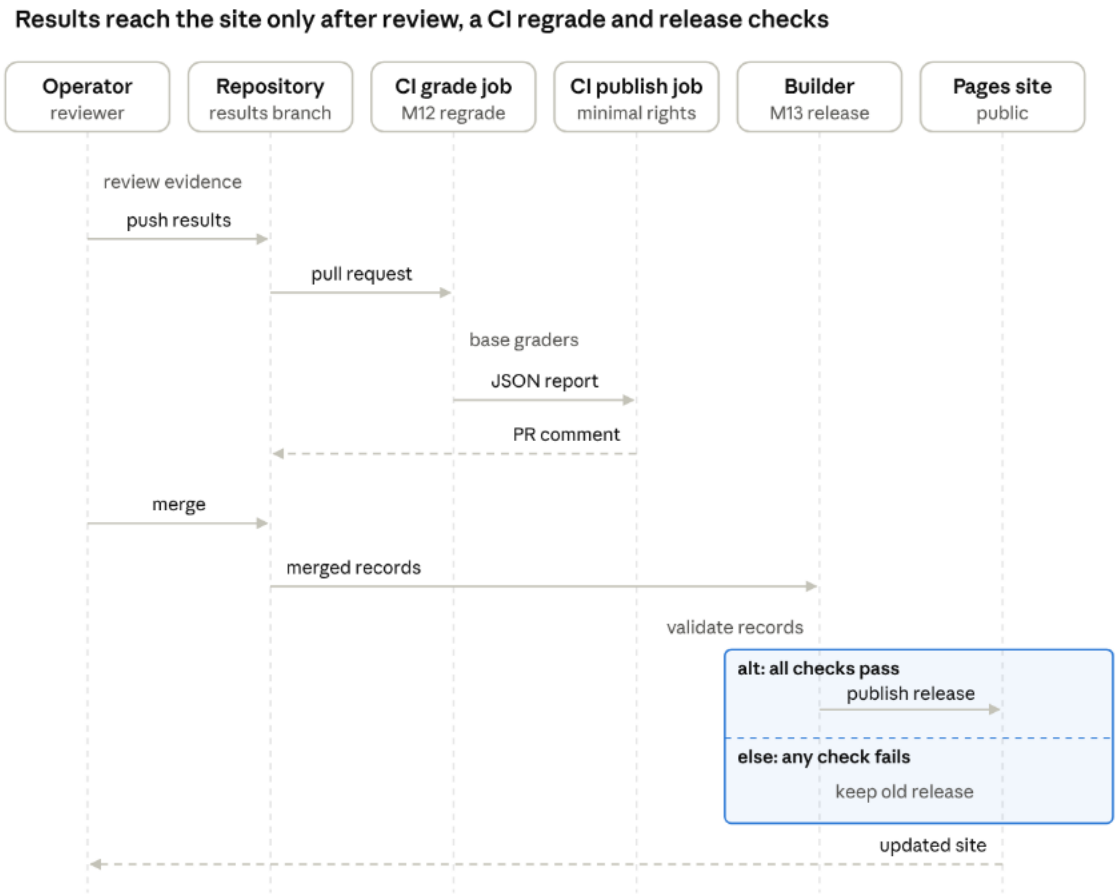


Figure 7. Sequence: review, CI regrade and publication (proposed)

An operator reviews evidence before pushing it to a results branch; the pull request is regraded by an unprivileged CI job that uses base-branch graders and treats candidate output as data, and a separate minimal-permission job posts the comment (ADR-11, Spec 4.5). After merge, the release builder validates records and publishes them to GitHub Pages within 2 minutes (Spec 4.4); if any check fails, the previous release stays live. Today's referee workflow still grades and comments in one job (see Security Considerations).

Native-tool pilot run (Approach B)

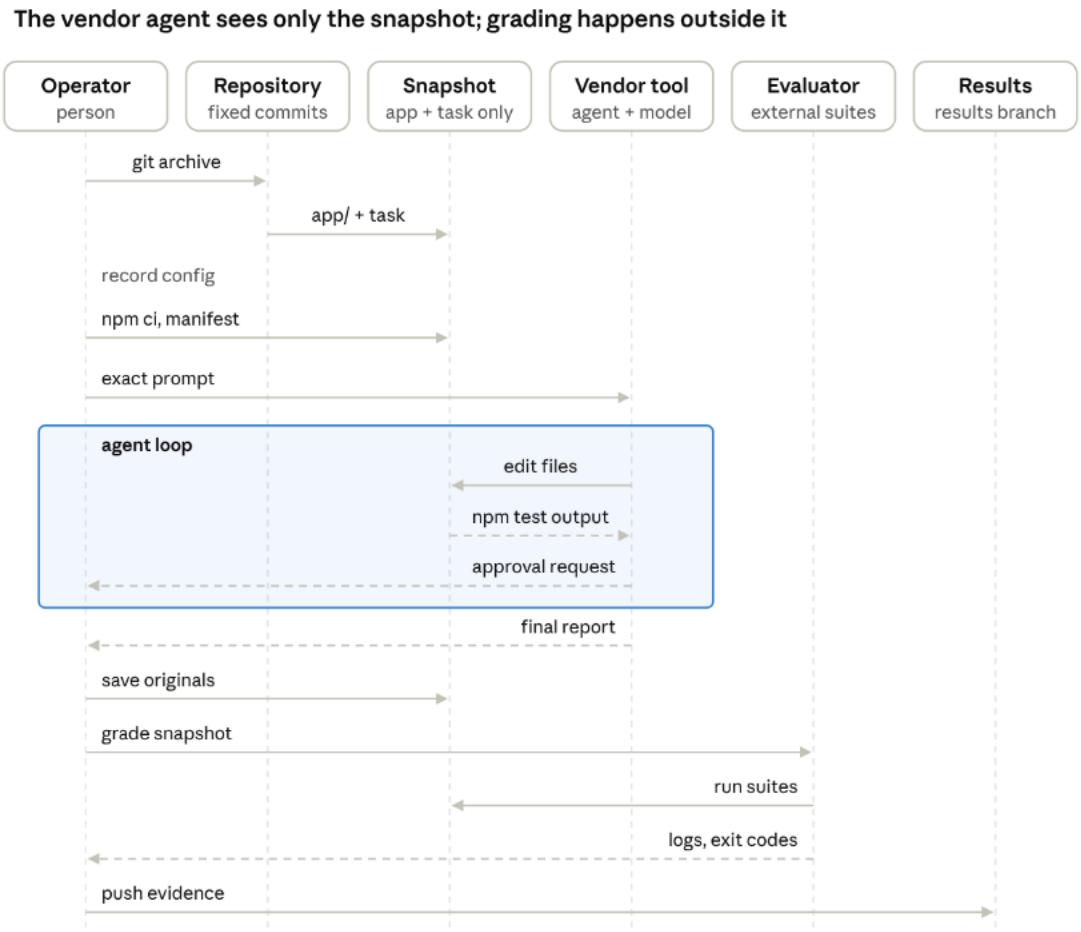


Figure 8. Sequence: one native-tool pilot run (Approach B)

The operator exports only `app/` and the task file from the fixed starting commit into a fresh folder, records the tool's configuration and grader hashes, and sends the exact prompt. The vendor tool works only inside that snapshot, asking the operator for approvals in its own way. When it reports, the operator saves the original files before any feedback, grades them with evaluator suites kept outside the snapshot, and pushes evidence to a results branch. This is the procedure behind all nine pilot runs in the Scenario view.

Development

The development view shows where each part of the system lives in the repository, how branches separate frozen inputs from results, and how code is built and tested.

Repository layout (master).

Path	Contents	Modules

app/	Node/Express todo testbed and its node:test suite	Testbed
baseline/	Clean copy of the testbed (5 passing tests, V02)	Testbed
pipeline/autogen/harness/	Harness Python package: tasks.py, model_profiles.py, staging.py, instrumented_client.py, orchestrator.py, executor.py, scoring.py, telemetry.py, pr_export.py, referee.py, plus verify_*.py self-checks	M01–M06, M09, M10, M12
pipeline/autogen/graders/	Functionality, security, quality and Node graders	M08
pipeline/autogen/run_benchmark.py	Command-line entry point	M02
pipeline/autogen/tests/	pytest suites for executor isolation and grader behaviour	Verification
pipeline/tasks/	Task prompts and manifests (001, 003, …)	M01 input
.github/workflows/	agent-pr-referee.yml, autogen-pipeline-ci.yml, publish-sandbox-image.yml	M12, CI
docs/, index.html, demo.html	Project documentation and the microsite prototype	M13 (prototype)
course_artifacts/	Course deliverables such as this DDD	None

Branch model. Inputs are frozen on dedicated branches so every attempt starts from an identical, recorded commit.

Branch	Role	Rule
master	Harness code, documentation and site prototype	Normal reviewed development
benchmark-start-v1 (be98d69), benchmark-start-v2-edit (4e258a9)	Frozen starting app and task for Task 001 and Task 003	Never modified after publication; agent output is never merged here
results/antigravity-delete-run01 (protocol folder), benchmark-protocol-v2-edit (0299644)	Evaluator suites, procedures and validation records	One agreed evaluator commit per task; changes create a new evaluator version
results/<tool>-<task>-run<NN>	Evidence for one attempt (README, manifests, logs, original files)	Evidence only; based on the starting or protocol commit
Documentation, AutoGen	Status reports and timesheets; harness experiments	Supporting work

Build and test.

- Testbed: `npm ci` from the committed lockfile, then `npm test` (Node 22 or 24).
- Evaluators: `node --test` API suites and Playwright 1.64.1 browser suites, installed outside any agent workspace.
- Harness: Python dependencies pinned in `pipeline/autogen/requirements.txt`; pytest suites and `verify_*.py` checks run in `autogen-pipeline-ci.yml`.
- Sandbox image: built and published by `publish-sandbox-image.yml`; runs pin the image digest.
- Referee: `agent-pr-referee.yml` regrades `agent/` branches on pull requests (currently targets `main`; see Security Considerations).

Scenarios (+1)

All four vendor tools passed every independent check on both pilot tasks; only one attempt needed a human correction. The two scenarios below exercise the views above end to end.

Scenario S1: Task 001, add a DELETE endpoint (API only).

1. Inputs are frozen at `benchmark-start-v1` (be98d69): the testbed without DELETE and the task file. The evaluator is a 9-test acceptance suite plus a 1-test leading-zero supplement (M01 contracts; Development view).
2. Each operator exports an isolated snapshot and runs their assigned tool with the exact prompt (Figure 8; physical view, operator workstation).
3. Original files are saved and graded outside the snapshot: local tests, then the two evaluator suites (Figure 6, freeze and grade steps).
4. Evidence is committed to `results/<tool>-delete-run01` (data flow step 5; Figure 7, first half).

S1 exposed a requirement gap: the original suite did not test leading-zero IDs. Antigravity's first attempt rejected `/api/todos/01` while passing 9/9, which led to the supplemental test, to ADR-12 (keep original and corrected attempts separately), and to the strict ID rule under Formulas & Algorithms.

Scenario S2: Task 003, edit a todo title (API and browser).

1. Inputs are frozen at `benchmark-start-v2-edit` (4e258a9). The evaluator commit 0299644 adds 12 API tests and 10 Playwright browser tests (UI-01 to UI-11).
2. The run follows Figure 8; three files may change (`server.js`, `test.js`, `public/index.html`).
3. Grading adds the browser suite, which checks accessible inline editing, literal rendering of HTML-like titles, error alerts, and keyboard use, the browser behavior the task file requires.
4. Grader hashes are compared across operators before coding (M01 rule); the only differences found were Windows CRLF versus Linux LF line endings.

Pilot results. Times were measured differently by each operator (UI display, event logs, stopwatch or process timer), so they show order of magnitude only and are not a speed ranking (A-4).

Task	Tool (operator)	Model, mode	Elapsed	Local tests	Evaluator results	Correction
001	Antigravity (Kapil)	Antigravity 3.8 Flash Medium, interactive IDE	≈ 6 min UI estimate	8/8	9/9 main; leading-zero check failed	1 prompt, ≈ 4 min; then 9/9 and `/01` → 204
001	Copilot CLI 1.0.95 (Ross)	Auto → GPT-6 Luna, interactive	73.2 s (event log)	8/8	9/9, 1/1	None; 3 approvals
001	Codex CLI (Joseph), run 01	GPT-6.1 Sol, low reasoning	129.1 s session	7/7	9/9, 1/1	None
001	Codex CLI (Joseph), run 02	GPT-6 Luna, low reasoning	47.4 s session	7/7	9/9, 1/1	None
001	Claude Code 2.1.296 (Bruno; AI operator)	Default Sonnet 5.5, headless	33.4 s (process timer)	8/8	9/9, 1/1	None
003	Antigravity (Kapil)	Antigravity 3.8 Flash Medium, interactive IDE	533.3 s	16/16	12/12 API, 10/10 browser	None
003	Copilot CLI 1.0.95 (Ross)	Auto → GPT-6 Luna, medium, interactive	122.9 s (event log); 205 s stopwatch	12/12	12/12 API, 10/10 browser	None; 8 approvals
003	Codex CLI (Joseph)	GPT-6.1 Sol, low reasoning, headless	72.6 s (stopwatch)	13/13	12/12 API, 10/10 browser	None
003	Claude Code 2.1.300 (Bruno; AI operator)	Default Sonnet 5.5, headless	87.3 s (process timer)	14/14	12/12 API, 10/10 browser (Chromium 141)	None

What the scenarios validate: frozen inputs, external grading, scope manifests and evidence publishing all work across four tools and three operating systems (Windows, macOS, Linux). What they do not yet validate: M09 scoring, the 120 s limit (Antigravity's runs exceed it), CI regrade, and the release builder.

System Configuration Details

Input	Behavior
Model profiles	local-dev (Ollama, default), azure-dev, azure-prod; only azure-prod is benchmark-valid
Limits	5 autoreplies, 15,000 tokens
Secrets	Host. env variables; never in YAML
--timeout	120 s per execution (SRS requires 60 s per command)
--network	Enables container networking; development only
--interactive	Real approvals; otherwise, five automatic approvals
--export-pr	Dry run unless --push; targets main although the default branch is master

Staging does not exclude `.env` files, which violates Spec 1.2 if a target contains one. Proposed: copy only manifest-listed files and define each cohort in a committed campaign manifest (commit, tasks, suite hash, image digest, profile, limits, spend cap, metric contract).

Component	Choice	Risk
Orchestration	AutoGen 0.7.x	In maintenance mode; Microsoft Agent Framework recommended [17]. Pin and isolate (ADR-05).
Isolation	Docker, network none	Shares host kernel [16]; limits must be set
Testbed	Node 22, Express 4.22	Pin lockfile
Records	JSON Schema 2020-12 [20]	Semantic checks still needed
Hosting and CI	GitHub Pages and Actions [19], [18]	Pin actions by commit SHA
Evaluators	node:test API suites; Playwright 1.64.1 browser suites	Browser build must match the pinned Playwright version

System Inputs API if used

No hosted API is used. Inputs are local CLI, task files, and model profiles.

```
python run_benchmark.py --task 001-add-delete-endpoint --model-profile azure-dev
```

The runner returns 0 after any completed run, even an unreliable one. Proposed exit codes: 0 passed, 1 failed, 2 invalid input or infrastructure error, 130 cancelled.

Contract	Current	Proposed
Task	ID, target, test command, framework, forbidden patterns; allowed files in prose	Machine-readable allowed files, suite ID, seed commit, hashes
Acceptance	npm test in the candidate's tree	Evaluator-owned suite with fixed test inventory
Model profile	Provider, deployment, benchmark_valid	Add version, settings hash, capability check
Tool profile	None	Tool, version, model, mode, approvals, integrations [8]

The proposed model adapter records every call, including failures, with usage marked complete or unknown. It retries at most twice and ends the run as `infrastructure_error` after the third failure (Spec 2.5.3).

System Output API if Used

The todo API is the application agents modify, not a harness service. State is in memory and resets on restart.

Endpoint	Current (`master`)	Task 001 (`benchmark-start-v1`)

GET /api/todos	200, todo array	Unchanged
POST /api/todos	201; 400 for missing or blank title	Unchanged
PUT /api/todos/:id	200; 404 if missing	Unchanged
DELETE /api/todos/:id	Already present; accepts 1abc via parseInt	Removed from seed; agent must add strict ID checks (400/204/404)

`master`'s DELETE passes 3 of 9 pilot acceptance tests (V04), so task 001 on `master` is not usable for scoring until the branch is merged (D06).

Outputs today: `logs/<task>_<run>.json` (schema 1) and a CI referee comment. Proposed: a schema 3 run record, grader evidence, an eligible patch, and a release bundle containing no prompts, secrets, or local paths (Spec 4.2.4).

System Decomposition

The system is a modular application on the operator host with untrusted work isolated in containers. Candidate code never reaches the score writer, credentials, or publication.

Module	Responsibility	Current code	Status	SRS
M01 Contracts	Validate task, profile, policy, campaign	tasks.py, model_profiles.py	Partial	2.3, 2.5
M02 Controller	Lifecycle, budgets, final status	run_benchmark.py	Proposed	2.1, 2.4, 2.5
M03 Staging	Copy seed, prepare dependencies	staging.py	Partial	1.2, 1.4
M04 Model adapter	Provider calls, usage, retries	instrumented_client.py	Partial	2.5, 4.2
M05 Coder loop	Turns, feedback, approvals	orchestrator.py	Partial	2.1, 2.2, 2.4
M06 Sandbox	Container limits, watchdog, teardown	executor.py, base.py	Partial	1.1, 1.3, 1.4
M07 Freezer	Hash candidate, enforce manifest	None	Proposed	2.3
M08 Evaluation	Acceptance suites, scans, quality	graders/	Partial	1.5, 4.1, 4.3
M09 Scoring	Dimensions, overrides, aggregation	scoring.py	Partial	Test Plan
M10 Record writer	Terminal record, atomic write	telemetry.py	Partial	4.2
M11 Assessment	Track A cells and review	Specification only [4]	Proposed	None
M12 Export and referee	Patch export, CI regrade	pr_export.py, referee.py	Partial	4.5
M13 Release builder	Publish records to site	index.html prototype	Proposed	4.4

System Coupling

Dependencies point from policy toward adapters (Figure 4). Scoring imports no Docker or provider code; publication depends only on validated records.

Boundary	Current	Proposed
Task to grader	Task supplies a shell test command run in the candidate tree	Evaluator-owned suite ID
Executor to Docker	Global patch of the Docker SDK create call	Explicit per-container specification
Loop to limits	10-message ceiling approximates 5 turns	Host counter of coder turns
Graders to workspace	Read-write mount of the agent workspace	Read-only frozen candidate
Scoring to tools	Tool error becomes a 0 score	Typed result; no score on tool failure
Export to Git	Commits scratch history to main	Apply patch onto upstream master

A new task revision, grader version, or model version creates a new cohort; earlier results are not mixed with it.

System Cohesion

Each module has one reason to change: runtime policy (M06), evaluation method (M08), formulas (M09), assessment rules (M11), or presentation (M13). Today `run_benchmark` mixes setup, grading, export, and record writing, so an export error can prevent the record from being written. The design separates these while keeping the existing `SandboxRunner` and `GraderResult` interfaces.

Application servers

There is no harness application server; the controller is a local Python process. The only server is the testbed Express process (port 3000; tests use 3001 or an ephemeral port). Each run starts a fresh process, which resets state (Spec 3.1.2). The image does not include Express and staging skips `node_modules`, so offline grading needs a prepared dependency layer.

Web servers

GitHub Pages serves the static site [19]. It runs no model code and has no backend. Spec 4.4 requires it to render results from committed records, label code as AI-generated, show model version and settings, and update within 2 minutes of a merge. The current `index.html` does not read records. WCAG 2.2 AA is the proposed accessibility target [23].

Database servers, design and Schemas

No database server is used (Spec 3.1 prohibits one for the testbed). Records are versioned JSON files validated with JSON Schema 2020-12 [20]; the release builder checks references between files.

Group	v1 (code)	v2 (SRS Appendix A)	v3 change (Proposed)

Identity	Task, run ID	Adds attempt 1–3	Adds campaign, approach, phase, retry_of
Outcome	None	Status (5 values), error, turn count	Unchanged
Agent and model	Model label, profile	Agent and model name, version, settings	Allows unknown version with reason
Timing	Wall time (stops before grading)	Adds active time	Adds approval, grading, cleanup time
Usage and cost	Tokens; missing usage stored as 0	Required integers; estimated cost	Null allowed when unknown
Scorecard	Five dimensions, reliable flag	Required scores, total_max, zeroed_by	Null scores for infrastructure_error

v2 cannot record an unknown token count or an unscored infrastructure failure, so v3 is proposed as an SRS amendment (D05). The SRS states the schema file `benchmark_run.schema.json` is committed, but it is not in the repository.

- A passed run has a null error; any other status needs an error.
- A non-null `zeroed_by` forces total 0 and status failed.
- A null token score forces `total_max` = 90.
- Per-call tokens must sum to totals; every hash must resolve.

Track A cells use the fields in the feature matrix specification [4]: 2 configurations × 18 features = 36 cells. “Not established from documentation” is a valid result, not a zero.

Storage Requirements

Current sizes are small (`app/` 24 KB, `pipeline/` 952 KB). Run record size is not yet measured; a planning estimate of 5 MiB per run gives about 480 MiB for 96 runs [6]. Raw logs stay private; only redacted records enter Git. Retention periods need a decision (D09).

Storage layout

Path	Contents	Access
pipeline/tasks/	Task prompts and manifests	Reviewed changes only
evaluator/<suite>/	Acceptance suites and hashes	Never mounted for the agent
schemas/	Record schemas	Data owner
Temporary run folder	Workspace, then frozen candidate	Deleted after the record is written
benchmark-results/<approach>/<run>/	Records, original and corrected files, test output	Written by host; reviewed before merge

site/data/<release>/	Published JSON	Release builder only

Files are written to a temporary path, validated, then renamed. Suite files are hashed after converting line endings to LF (V05), which also removes the Windows/Linux hash differences seen in the pilots.

Module Specifications

Module	Processing	Failure handling	Verification
M01 Contracts	Validate inputs and compute hashes before any run starts	Invalid input stops the run before an ID is assigned	Malformed and out-of-range inputs
M02 Controller	Assign run ID; check turn, time, token, spend, and rate budgets before each step	A finalizer always writes one record; one rerun for infrastructure errors	TC-2.1-x, TC-2.5-x
M03 Staging	Copy manifest files only; install from lockfile; record tree hash	Reject .env and links outside the target	TC-1.2-3, TC-1.4-2
M04 Model adapter	Record every call with latency and usage completeness	Two retries, then infrastructure_error	TC-2.5-3, TC-4.2-3
M05 Coder loop	Count turns; return output truncated to 2,000 characters; request approval after two failed turns	Approval expires after 180 s; run aborted	TC-2.2-x, TC-2.4-x
M06 Sandbox	Network none, non-root, 2 CPUs, 2 GB, 128 processes, 60 s watchdog	Kill and remove containers on every path	TC-1.1-x to TC-1.4-x
M07 Freezer	Stop agent, snapshot, hash, compare with seed	Out-of-manifest change: failed, total 0	TC-2.3-x
M08 Evaluation	Run evaluator suites, secret scan, license scan, quality checks	Missing tool or report: infrastructure_error	TC-1.5-x, TC-4.1-x, TC-4.3-x
M09 Scoring	Apply formulas, zero override, and aggregation	Refuse to score if a required dimension is missing	Formula fixtures
M10 Record writer	Validate and write one record atomically	On failure, block export and publication	TC-4.2-x
M11 Assessment	Record each cell with a primary source; second reviewer	No source: “Not established”	All 36 cells
M12 Export and referee	Apply patch onto upstream; grade in CI from base branch	Unsafe history or failed checks: no export	TC-4.5-x
M13 Release builder	Validate and publish records with AI label and metadata	Any failed check keeps the previous release	TC-4.4-x

Module Design

Contracts and results are immutable in Python dataclasses. Adapters return typed outcomes instead of raising expected conditions. One scored run executes at a time; parallel runs wait until the global Docker patch is removed, and budget reservation is atomic.

Testbed: `baseline/` passes its 5 tests (V02); `app/` fails because of a stray first line (V03), but SRS TC-3.3-1 names `app/` as the clean baseline (D06). Task 001 on `master` describes a DELETE route that already exists; `benchmark-start-v1` fixes this. Tasks T3–T8 from the SRS test plan is not yet written.

Module Processing

1. Validate the campaign, task, profile, limits, and host.
2. Assign a run ID and stage the task without credentials.
3. Start the container and confirm its limits.
4. Before each model calls, check budgets; record the call.
5. Request approval when Spec 2.4 requires the active clock pauses.
6. Run each command under the 60 s watchdog; return sanitized output.
7. When finished or out of budget, freeze the candidate and check the manifest.
8. Grade with evaluator suites and scanners.
9. Score, clean up, and write the record.

Event	Status
Secret or forbidden action	failed, total 0 (takes precedence)
Provider, environment, or tool failure	infrastructure_error (one rerun)
Active time reaches 120 s	timeout
Approval rejected or expired	aborted
Token limit reached	failed (token_limit_exceeded)
Work completed but not accepted	failed
All checks pass	passed

Interfaces

ID	From → to	Contract	Rule
I01	M01 → M02	Validated inputs with hashes	Nothing runs until valid
I02	M02 → M04	Messages, request ID, token reservation	Every call recorded
I03	M05 → M06	Approved command	Output bounded
I04	M06 → M05	Exit code and output (≤ 2,000 characters)	Secrets redacted first
I05	M07 → M08	Frozen candidate ID	Read-only
I06	M08 → M09	GraderResult: suite hash, counts, findings	Missing report = unscored
I07	M09 → M10	Scores and total_max	Range-checked
I08	M10, M11 → M13	Validated public records	Secrets and mixed cohorts rejected
I09	Tool adapter → M02	Capability report: headless, model pinning, sandbox routing, usage	Missing capability = supervised only

Interfaces are in-process calls that pass immutable dataclasses (Module Design); payloads that persisted (I06–I08) are JSON validated against JSON Schema 2020-12 [20]. Only I02 (the provider call, HTTPS from the host) and I08 (publication to GitHub) cross a network. Expected errors are typed results rather than exceptions, so every failure maps to one status in the table above.

Formulas & Algorithms

Metric (weight)	Implemented	SRS v1.1
Functionality (35%)	Pass rate of the candidate's own tests	Pass rate of evaluator acceptance tests
Security (25%)	100 if scans are clean, else 0	100 − 25 per new high or critical finding; secret sets total to 0
Quality (20%)	Python-oriented checks; runtime within 2× baseline	Formatting, complexity ≤ 10, duplication, runtime within 1.2×
Tokens (10%)	100 × (1 − U/L)	100 up to 0.5L, then linear to 0 at L
Human help (10%)	−10 per correction	−25 per avoidable intervention, peer-rated

```
total = 0.35F + 0.25S + 0.20Q + 0.10T + 0.10H
T = 100 if U <= 0.5L, else max(0, 200 * (1 - U/L))
cost = sum(U_in * P_in + U_out * P_out) / 1,000,000
```

If any agent lacks token counts, T is dropped and totals are out of 90. Scores are averaged over three attempts per task, then across tasks, with min–max ranges and an instability flag. Example: 7,500 of 15,000 tokens scores T = 50 in the code and 100 under the SRS.

Task 001 ID rule: accept only digits representing a positive safe integer (`01` valid; `0`, `-1`, `1.5`, `1abc` return 400). `parseInt` alone is insufficient.

Architecture Decision Records

ADR	Decision	Alternative rejected	Status
01	Keep each comparison approach in a separate cohort	One combined leaderboard	Proposed
02	Local modular application with containers	Microservices or hosted queue	Proposed
03	Enforce limits on the host, not through prompts	Prompt instructions as controls	Partial
04	Grade with evaluator-owned acceptance suites	Agent's own tests	Used in pilot
05	Keep AutoGen behind an adapter; evaluate Microsoft Agent Framework later [17]	Immediate migration	Proposed
06	Record schema v3 as an SRS amendment	Relabel v1; store unknowns as 0	Proposed
07	Version the metric contract	Silent renormalization	Proposed
08	Wrap the model client instead of a LiteLLM gateway	Extra proxy process	Implemented
09	JSON files and static site	Database server	Proposed
10	Export patches onto upstream history	Push scratch history	Proposed
11	Separate CI grading and publishing jobs; pin actions by SHA [18]	Grade with pull-request code	Proposed
12	Keep original and corrected attempts separately	Overwrite the original	Used in pilot

13	Run native tools through a shared-evaluator procedure (Approach B), scored separately from the harness	Fold every tool into the harness now	Used in pilot

ADR-02: Local modular application with containers

- **Context:** The harness must run on the operator's machine (C-1). A full campaign is 96 runs, about 3.2 hours of agent time, on a $100 budget (C-3) and a one-semester schedule (C-4).
- **Options:** (a) a modular monolith with Docker containers for untrusted code; (b) microservices for controller, grader and publisher behind a queue; (c) a hosted cloud service.
- **Trade-off:** Options b and c scale horizontally and isolate each service, but add network contracts, deployment, cloud credentials and running cost, none of which this workload needs. Option a keeps interfaces in-process, typed and easy to test; it limits parallelism to one machine and shares one fault domain, while containers still isolate the untrusted code.
- **Consequence:** One host process runs one scored run at a time. If campaigns outgrow a machine-day, the fallback is an Azure VM per A-1, not a re-architecture.

ADR-05: Keep AutoGen behind an adapter

- **Context:** The harness loop and executor are built on AutoGen 0.7.x, which is in maintenance mode; Microsoft recommends Agent Framework [17].
- **Options:** (a) keep AutoGen, pinned, behind the M05 and M06 interfaces; (b) migrate to Agent Framework now; (c) write a custom agent loop.
- **Trade-off:** Option b aligns with the sponsor's ecosystem and long-term support, but would mean re-validating sandbox and grader integration mid-semester. Option c gives full control but re-implements tested code. Option a carries maintenance risk, limited by pinning and the adapter boundary.
- **Consequence:** Only M05 and M06 import AutoGen, so a later migration replaces one adapter. Migration is evaluated after Dec. 4 or immediately if a blocking defect appears.

ADR-08: Wrap the model client instead of a LiteLLM gateway

- **Context:** Every model call's usage, latency and failure must be recorded (Spec 2.5.3, 4.2). The SRS view proposed a LiteLLM gateway.
- **Options:** (a) an in-process wrapper around the model client (`instrumented_client.py`); (b) a LiteLLM proxy process; (c) provider dashboards.
- **Trade-off:** A gateway is provider-neutral and could also meter native tools that accept a base-URL override, but it adds a process, a port, configuration and a failure mode. The wrapper is simpler and adds no secret surface, but covers only harness calls. Dashboards cannot attribute usage to a run.

- **Consequence:** Implemented for Approach C. Approach B relies on tool-reported usage (A-7); the gateway is reconsidered only if a tool supports base-URL override (A-2).

ADR-09: JSON files and a static site instead of a database server

- **Context:** Up to 96 records of about 5 MiB each must be reviewable in Git and published without a backend (C-7, Spec 4.4); Spec 3.1 prohibits a database for the testbed.
- **Options:** (a) versioned JSON files, JSON Schema and a static site; (b) an SQLite file; (c) a hosted database and API.
- **Trade-off:** Option c offers queries and concurrent writers, but adds hosting cost, credentials and attack surface, and its records cannot be diffed in a pull request. Option b is queryable but reviews poorly as a binary file in Git. Option a gives diffable, hash-addressable records with no runtime to secure, at the price of weak querying and integrity checks that the validator must enforce.
- **Consequence:** The release builder performs cross-file checks and computes aggregates at build time. Past a few thousand records, an SQLite file generated at release time is the next step.

ADR-11: Separate CI grading and publishing jobs

- **Context:** Today's referee builds graders from the pull request and comments from the same job, with write permission (E15), while pull requests contain agent-generated, untrusted code [18].
- **Options:** (a) keep one job; (b) an unprivileged grading job using base-branch graders, then a separate minimal-permission job that publishes a bounded JSON report; (c) no CI regrade.
- **Trade-off:** Option b costs extra job time and artifact plumbing against the 120 s CI target (Spec 4.5), but untrusted code never runs with a write token and cannot alter its own graders. Option a is simpler but exploitable; option c loses independent verification.
- **Consequence:** Figure 7 shows the proposed flow. Actions are pinned by commit SHA, and the publishing job never executes report contents.

ADR-13: Native tools run through a shared-evaluator procedure

- **Context:** The sponsor's question is how each vendor pipeline performs in its shipped defaults [3]. IDE-only tools cannot meet the harness's non-interactive constraint (C-6), and the pilots had to start before the harness was complete.
- **Options:** (a) add every tool to the harness through adapters now; (b) run native tools through a written procedure that shares evaluators, evidence layout and publication, keeping harness scoring for Approach C; (c) drop the harness and run native tools only.
- **Trade-off:** Option a gives uniform scoring, but tools lack headless mode or usage reporting (A-2), which would delay all results. Option c loses enforced limits and controlled comparisons. Option b answers the sponsor now with identical grading, at the cost of manual steps, operator variance and timing that is not comparable across operators.

- **Consequence:** Results stay in separate cohorts (ADR-01). A tool joins Approach C only through I09. The team should standardize one timing method for later campaigns.

SRS Traceability Matrix

Every SRS feature (F1–F5) and specification section referenced in this document maps to at least one module, interface and test. The last column records what is verified today.

SRS requirement	Spec sections	Modules	Interfaces	Verification	Evidence today
F1 Standard reference application with automated tests	3.1, 3.2, 3.3	Testbed (app/, baseline/); M03 Staging	I01	TC-3.3-1; testbed suite	Baseline 5/5 (V02); Task 001 and 003 starting suites pass (4/4, 9/9)
F2 Isolated, credential-free execution	1.1, 1.2, 1.3, 1.4	M03 Staging; M06 Sandbox	I03, I04	TC-1.1-x to TC-1.4-x; TC-1.2-3	Network-none containers implemented; CPU, memory and process limits not yet enforced; Approach B uses snapshot isolation
F3 Orchestration under turn, time, token and cost limits	2.1, 2.2, 2.4, 2.5	M02 Controller; M04 Model adapter; M05 Coder loop	I01, I02, I09	TC-2.1-x, TC-2.2-x, TC-2.4-x, TC-2.5-x	Partial (10-message ceiling approximates 5 turns); approval timeout not implemented
F4 Independent grading, secret and license scans	1.5, 2.3, 4.1, 4.3	M07 Freezer; M08 Evaluation; M09 Scoring	I05, I06	TC-2.3-x, TC-1.5-x, TC-4.1-x, TC-4.3-x; formula fixtures	Evaluator-owned suites used in all 9 pilot runs; freezer done manually by manifests; scans and M09 scoring not yet applied to pilots
F5 Per-run telemetry and public microsite	4.2, 4.4, 4.5	M10 Record writer; M12 Export and referee; M13 Release builder	I07, I08	TC-4.2-x, TC-4.4-x, TC-4.5-x	Evidence published on results branches; site prototype does not read records; CI referee targets the wrong branch
Sponsor: compare native pipelines in shipped defaults	C-9 [3]	Approach B procedure; M08; M10 record format	I09 (promotion path)	Procedure checklists [8], [24]	9 runs across 4 tools on 2 tasks (Scenario view)

Sponsor: documented feature comparison	Feature matrix spec [4]	M11 Assessment; M13	I08	All 36 cells reviewed	Specification v0.2; cells not yet recorded
Legal L1–L5	SRS legal items	M02 (spend), M08 (license), M06 (limits), M13 (AI label)	I02, I06, I08	See Security Considerations	Mapped; spend caps and license scans not yet run

Architecture Decision Records

ADR	Decision	Alternative rejected	Status
01	Keep each comparison approach in a separate cohort	One combined leaderboard	Proposed
02	Local modular application with containers	Microservices or hosted queue	Proposed
03	Enforce limits on the host, not through prompts	Prompt instructions as controls	Partial
04	Grade with evaluator-owned acceptance suites	Agent's own tests	Used in pilot
05	Keep AutoGen behind an adapter; evaluate Microsoft Agent Framework later [17]	Immediate migration	Proposed
06	Record schema v3 as an SRS amendment	Relabel v1; store unknowns as 0	Proposed
07	Version the metric contract	Silent renormalization	Proposed
08	Wrap the model client instead of a LiteLLM gateway	Extra proxy process	Implemented
09	JSON files and static site	Database server	Proposed
10	Export patches onto upstream history	Push scratch history	Proposed
11	Separate CI grading and publishing jobs; pin actions by SHA [18]	Grade with pull-request code	Proposed
12	Keep original and corrected attempts separately	Overwrite the original	Used in pilot

System Size and Performance

The SRS campaign is 96 harness runs. At 120 s each, active agent time is at most 3.2 hours, excluding grading and reruns. No scored harness run has been timed yet; the pilot evidence below comes from Approach B runs and grading.

Target	Source	Current evidence
≤ 2 CPUs, 2 GB, 128 processes	Spec 1.3	Not enforced
Command killed at 60–61 s	Spec 1.3	Not implemented (120 s default)
Sandbox ready < 3 s; removed < 5 s	Spec 1.4	Not measured

≤ 5 turns, ≤ 120 s active	Spec 2.1	Partial
Approval expires at 180 ± 1 s	Spec 2.4	Not implemented
Test suite < 500 ms	Spec 3.1	318 ms, one local sample (V02)
UI update ≤ 100 ms	Spec 3.2	Not measured
Telemetry overhead < 5%	Spec 4.2	Not measured
Site updated ≤ 2 min after merge	Spec 4.4	Not connected
CI grading ≤ 120 s	Spec 4.5	Not measured

Each result records the host profile (CPU, RAM, OS, Docker version, image digest). Results from different platforms are not combined.

Capacity plan. One scored run executes at a time (Module Design). The 96-run campaign needs about 3.2 hours of agent time plus grading; with about 10 s of grading per run, that is roughly 3.5 hours on one workstation. Model cost is bounded by the per-trial token cap within the $100 credit (C-3). Approach B token usage ranged from about 113,000 to 193,000 input tokens per Task 001 attempt where reported, most of it cached.

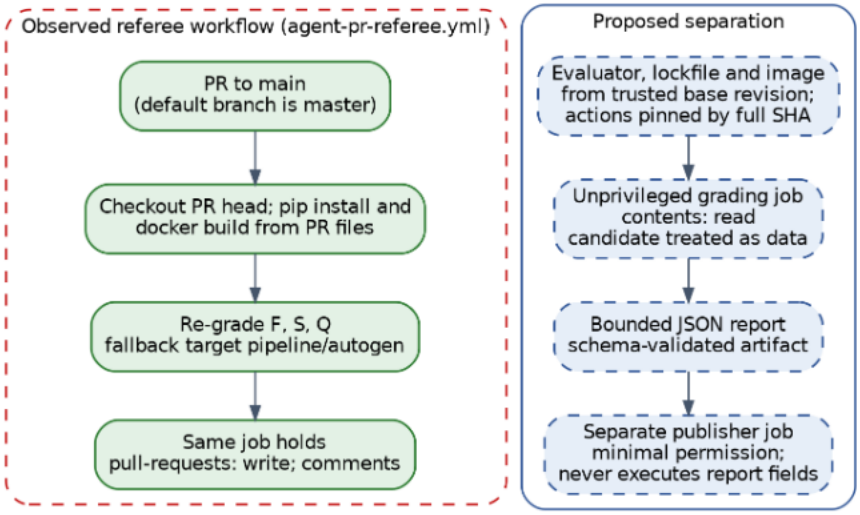
Security Considerations

Security is assessed as a property of the pipeline, not the model. Whether a secret leaks depends on the harness, its MCP tools and connectors, and the defaults it ships with, so each tool is first scored in its out-of-box configuration. This shows whether a pipeline is tuned for production-grade work or only for prototyping [3].

Table 21. Security Controls

Threat	Spec	Control
Secret or credential exposure	Spec 1.2, 4.1	Credential-free staging; secret scan; any leak sets the total to 0
Prompt injection in task or repository content [13]	Spec 1.1, 2.3	No network; manifest-listed files only; limits enforced on the host
Over-broad tool approvals	Spec 2.4	Record the approval mode (interactive or autopilot) and every approval
Sandbox escape or resource abuse [16]	Spec 1.3	Non-root container; 2 CPUs, 2 GB, 128 processes; 60s watchdog [14]
Tampering with grading	Spec 2.3, 4.3	Evaluator-owned suites outside the workspace; read-only frozen candidate
Leaks through published results	Spec 4.2.4	Redacted records with no prompts, secrets, or local paths

In Approach B the vendor tool's own sandbox replaces M06, so the operator records its approval and sandbox settings and any access outside the snapshot. One pilot observed an agent writing scratch files to its harness-provided temporary folder outside the snapshot; the evidence preserves them.



The referee workflow triggers on `main`, but the default branch is `master`. It builds graders from the pull request and comments in the same job (E15). The proposed workflow grades with base-branch code in an unprivileged job and publishes from a separate job (Spec 4.5).

Legal items L1–L5 in the SRS map to these controls: spend and rate limits (L1), license scanning (L2), container limits (L3), dependency notices (L4), and redacted records with AI labeling (L5). Risk management follows the NIST AI RMF [12] functions: risks are mapped in the assumptions and component-risk tables, measured by the evaluators, and managed by the ADRs above.

Public Health, Safety and Welfare

The design meets the specified need; the sponsor needed an objective, repeatable way to compare AI coding pipelines, and the architecture delivers fixed inputs, evaluator-owned grading and published evidence, already exercised in nine pilot runs across four tools. It does so while containing the main risk to the public, which is untrusted AI-generated code, and it weighs the global, cultural, social, environmental, and economic factors below.

Factor	Consideration	Design response	Where
Public health and safety	AI-generated code can be unsafe: it may leak secrets, misuse resources or contain vulnerabilities, and software defects can harm the people who rely on a system	Agent code is never deployed (C-5); it runs only in network-less, non-root, resource-limited containers or isolated snapshots; secret and license scans grade every candidate, and a leaked secret scores zero	C-5; M06; M08; Security Considerations
Public welfare	Organizations may adopt AI coding tools on marketing	Independent, repeatable evidence of how each pipeline behaves in its shipped defaults,	Benchmark Approaches; ADR-

	claims and ship insecure software to their users	including security; limitations are stated, and no single run is presented as a general winner	01; A-4; Scenario view
Global	Tools and models come from several vendors and regions; availability and pricing change by region and over time; readers worldwide	Four vendors compared on identical inputs; runs on Windows, macOS and Linux; model version recorded per run and a new version starts a new cohort; the public site targets WCAG 2.2 AA accessibility [23]	A-6; Web servers; Scenario view
Cultural	Engineering practice is shifting from writing code by hand to supervising agents [3]; academic integrity requires clear disclosure of AI use	Human oversight is kept and measured (approvals and corrections are recorded, not hidden); AI-written code is labeled as AI-generated on the site; AI-drafted sections of this document are tagged	Formulas (human help); Spec 4.4; AI Impacts and Uses
Social	Privacy of students and sponsor staff; fair treatment of each vendor; trust in published results	No personal data in results and least-privilege access (C-7, FERPA); every tool gets the same task, prompt and evaluator; evidence is preserved so others can check it	C-7; A-5; Data view; Development view
Environmental	Model calls consume compute and energy, and repeated benchmark runs multiply it	Hard per-trial token caps and a bounded 96-run campaign; tokens are recorded per run as a proxy for compute use; one run at a time on existing workstations, with no dedicated servers; the static site needs no always-on backend (ADR-09)	C-3; M02; System Size and Performance
Economic	Tool subscriptions and model usage cost money; generated code can carry license liability	Harness spend capped at the $100 sponsor credit; cost per run computed from recorded tokens; open-source components (Docker, Node.js, Playwright, JSON Schema); license scanning of candidates (L2) [22]	C-3; Formulas & Algorithms; Security Considerations

Limits of this assessment. Energy use is estimated from token counts, not measured. Tasks are English-language and limited to one JavaScript application (C-2), so results may not hold for other languages, domains or safety-critical software. The pilot sample is small, and the team knew one edge case in advance, so pilots are not fully blind (A-4).

References

[1] Microsoft 2 Team 3, “Microsoft-Capstone,” GitHub repository, Oct. 2026. [Online]. Available: https://github.com/brunor560/Microsoft-Capstone

[2] J. Bressani, K. Srikanth, B. Rocha, and R. Volenec, “System Requirements Specifications,” ver. 1.1, Univ. of South Florida, Sept. 29, 2026.

[3] C. Hage and Microsoft 2 Team 3, sponsor meeting transcripts and notes, Sept. 4, Sept. 25, Oct. 2, and Oct. 9, 2026.

[4] R. Volenec, “Agentic Developer Tool and Pipeline Feature Matrix Specification,” ver. 0.2, Oct. 2026, in [1].

[5] R. Volenec, “Detailed Design Document,” draft ver. 0.1, Oct. 8, 2026, in [1].

[6] K. Srikanth, “Detailed Design Document,” ver. 0.1, Oct. 9, 2026, in [1].

[7] Microsoft 2 Team 3, “Detailed Design Document,” ver. 0.2, Oct. 9, 2026.

[8] K. Srikanth, “DELETE endpoint pilot: shared team procedure,” branch results/antigravity-delete-run01, Oct. 7, 2026, in [1].

[9] Microsoft 2 Team 3, “Team Status Report and Timesheet, Week 3,” branch Documentation, in [1].

[10] Software, Systems and Enterprise: Architecture Description, ISO/IEC/IEEE 42010:2022, 2022.

[11] P. Kruchten, “The 4+1 view model of architecture,” IEEE Software, vol. 12, no. 6, pp. 42–50, Nov. 1995.

[12] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023.

[13] OWASP, “LLM01:2025 Prompt Injection.” [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[14] Docker, “Resource constraints.” [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/

[15] Docker, “None network driver.” [Online]. Available: https://docs.docker.com/engine/network/drivers/none/

[16] Docker, “Docker Engine security.” [Online]. Available: https://docs.docker.com/engine/security/

[17] Microsoft, “AutoGen.” [Online]. Available: https://github.com/microsoft/autogen

[18] GitHub, “Secure use reference.” [Online]. Available: https://docs.github.com/en/actions/reference/security/secure-use

[19] GitHub, “What is GitHub Pages?” [Online]. Available: https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

[20] JSON Schema, “Draft 2020-12.” [Online]. Available: https://json-schema.org/draft/2020-12

[21] Software and Systems Engineering: Software Testing, Part 11: Guidelines on the Testing of AI-Based Systems, ISO/IEC TR 29119-11:2020, 2020.

[22] AboutCode, “ScanCode Toolkit documentation.” [Online]. Available: https://scancode-toolkit.readthedocs.io/

[23] W3C, “Web Content Accessibility Guidelines (WCAG) 2.2.” [Online]. Available: https://www.w3.org/TR/WCAG22/

[24] K. Srikanth, “Shared title-edit benchmark procedure” and “Title-edit evaluator validation,” branch benchmark-protocol-v2-edit, commit 0299644, Oct. 14, 2026, in [1].

[25] Microsoft 2 Team 3, pilot results branches results/<tool>-delete-run01 and results/<tool>-edit-run01 (Antigravity, Copilot, Codex, Claude Code) and results/codex-cli, Oct. 7–15, 2026, in [1].

Appendix A: Acronyms

Acronym	Meaning
ADR	Architecture Decision Record
AI	Artificial Intelligence
API	Application Programming Interface
C4	Context, Containers, Components, Code (diagram model)
CI	Continuous Integration
CLI	Command-Line Interface
CRLF / LF	Carriage return + line feed (Windows) / line feed (Linux, macOS) line endings
DDD	Detailed Design Document
FERPA	Family Educational Rights and Privacy Act
HLA	High-Level Architecture
HTTP(S)	Hypertext Transfer Protocol (Secure)
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
MCP	Model Context Protocol
NFR	Non-Functional Requirement
NIST AI RMF	National Institute of Standards and Technology AI Risk Management Framework
OWASP	Open Worldwide Application Security Project
PR	Pull Request
SHA / SHA-256	Secure Hash Algorithm (Git commit ID / file hash)
SRS	System Requirements Specification
TC	Test Case (SRS test plan identifier)
UI	User Interface
UML	Unified Modeling Language
USF	University of South Florida
VM	Virtual Machine
WCAG	Web Content Accessibility Guidelines

Appendix B: Glossary of Terms

Term	Definition
Approach A / B / C	The three comparison methods: documentation feature matrix, native-tool pilots, and the AutoGen harness (Benchmark Approaches)
Baseline (starting) commit	The fixed commit whose `app/` and task file every attempt starts from, e.g. be98d69 or 4e258a9
Candidate	The code an agent produced in one attempt
Cohort	A set of results that share task revision, evaluator version, model version and approach; results are compared only within a cohort
Correction	A human feedback prompt after the original attempt; recorded and graded separately (ADR-12)
Evaluator (grader)	Test suites and scans owned by the team, kept outside the agent workspace, that decide whether a candidate meets the task
Evaluator (protocol) commit	The agreed commit from which evaluator files are exported, e.g. 0299644
Frozen candidate	A read-only, hashed snapshot of the candidate taken before grading
Harness	The automated benchmark system described in this document (Approach C)
Manifest	A list of every file in a run folder with its SHA-256 hash, taken before and after the attempt
Native tool	A vendor's own coding agent (CLI or IDE) used in its shipped configuration
Operator	The person who runs an attempt and preserves its evidence
Results branch	A Git branch holding the evidence for one attempt only
Run record	The single validated JSON record written for each run (M10)
Snapshot	The isolated folder containing only `app/` and the task file, created with `git archive`
Supplemental test	An evaluator test added after a requirement gap was found (the Task 001 leading-zero check)

Appendix C: Documents, type and source

Document	Type	Source
System Requirements Specification v1.1 [2]	Word / PDF	Team 3 course deliverable

Agentic Developer Tool and Pipeline Feature Matrix Specification v0.2 [4]	Markdown	Repository [1], master
DELETE endpoint pilot: shared team procedure [8]	Markdown	Branch results/antigravity-delete-run01
Shared title-edit benchmark procedure and evaluator validation [24]	Markdown	Branch benchmark-protocol-v2-edit
Task files 001-add-delete-endpoint and 003-edit-todo-title	Markdown	Branches benchmark-start-v1, benchmark-start-v2-edit
Pilot results and evidence [25]	Markdown, CSV, logs, JSON transcripts	Results branches in [1]
Sponsor meeting transcripts and notes [3]	Teams transcripts	Sponsor meetings, Sept.–Oct. 2026
Team status reports and timesheets [9]	Markdown	Branch Documentation
Testing guidance for AI-based systems [21]	Standard	ISO/IEC TR 29119-11:2020